OpenStack and private cloud

Run a private cloud: Keystone, Nova, Neutron, Cinder and Glance, quotas, scheduling, networking and day-two operations.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What OpenStack is and its core services (14 min)
2. Identity, projects and quotas (13 min)
3. Compute: flavors, images and scheduling (14 min)
4. Networking: Neutron, security groups and floating IPs (15 min)
5. Operating OpenStack: logs, upgrades and troubleshooting (15 min)
6. OpenStack operations: "No valid host" and scheduling failures (13 min)
7. OpenStack operations: Neutron and Ceph troubleshooting (13 min)
8. OpenStack operations: maintenance, evacuation and a day-two runbook (12 min)
9. Cinder, Glance and OpenStack storage (12 min)
10. Horizon, Heat and OpenStack automation (12 min)
11. Deploying OpenStack with Kolla-Ansible (13 min)
12. OpenStack monitoring and capacity planning (12 min)
Cheat sheet

Lesson 1. What OpenStack is and its core services

14 minutes. See how OpenStack builds a private cloud from separate services, and how each one maps to something you know from AWS.

OpenStack is open-source software that turns a rack of servers into a private cloud. Teams get virtual machines, networks and disks on demand through an API, much like a public cloud, but on hardware the company owns. It suits organisations that need to keep data in their own buildings, control cost at large scale, or follow rules about where workloads run.

What you will learn

- The core services and what each does
- Control plane and compute nodes
- The two things every service depends on: a database and a message queue
- How OpenStack names map to AWS

The core services

Service	Name	What it does	Closest AWS idea
Identity	Keystone	Users, projects, roles, tokens, service catalog	IAM
Compute	Nova	Creates and manages virtual machines	EC2
Networking	Neutron	Networks, subnets, routers, security groups, floating IPs	VPC
Image	Glance	Stores VM images	AMIs
Block storage	Cinder	Volumes that attach to VMs	EBS
Object storage	Swift	Objects over HTTP	S3
Orchestration	Heat	Templates that build stacks	CloudFormation
Dashboard	Horizon	Web interface	Console
Placement	Placement	Tracks resources and picks hosts	Part of the scheduler

Every service has a REST API, and all of them accept the same token from Keystone. The `openstack` command line client talks to those APIs, and so does the dashboard.

Control plane and compute

- **Controller nodes** run the API services, schedulers, the database and the message queue. They are the **control plane**. Run three for high availability.
- **Compute nodes** run the hypervisor (usually KVM through libvirt) and the Nova compute agent. This is where virtual machines run.
- **Storage nodes** serve volumes (often Ceph) or objects.
- **Network nodes** (or the compute nodes themselves) run the Neutron agents.

What they all depend on

Two pieces of shared infrastructure sit under almost every service:

- A **database** (MariaDB or MySQL) that stores state: which servers exist, which networks, which users.

MARGA by EknathaLabs marga.eknathalabs.com

- A **message queue** (RabbitMQ) through which services send each other requests. When you ask Nova to boot a server, the API writes to the database and sends a message to the scheduler, which sends one to a compute node.

If the queue or database is unhealthy, many services fail together, with strange symptoms. When many different services show errors at once, look at these two first.

What a first request looks like

```
$ openstack token issue -c expires -c project_id
+------------+----------------------------------+
| Field      | Value                            |
+------------+----------------------------------+
| expires    | 2026-11-03T18:00:00+0000         |
| project_id | 4b1c8a0f6d5e4f2bb0a1c2d3e4f5a6b7 |
+------------+----------------------------------+
$ openstack server list
+--------------------------------------+--------+--------+---------------------+-------+---------- \
    +
| ID                                   | Name   | Status | Networks            | Image | Flavor    \
    |
+--------------------------------------+--------+--------+---------------------+-------+---------- \
    +
| 3f2a...                              | web-1  | ACTIVE | private=10.0.0.12   | ubuntu| m1.small  \
    |
+--------------------------------------+--------+--------+---------------------+-------+---------- \
    +
```

A successful `token issue` proves that Keystone works and your credentials are right. If that fails, nothing else will work.

Why choose it, and what it costs

A private cloud gives control and predictable cost at scale. The price is operations: you run the control plane, plan upgrades every six months or so, and manage hardware. Many teams use a distribution or deployment tool (Kolla-Ansible, OpenStack-Ansible, vendor products) to reduce that work.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How What OpenStack is and its core services fits together

Part	What it does	What goes wrong
The core services	Every service has a REST API, and all of them accept the same token from Keystone.	Treating each service as independent and ignoring the queue and database.
Control plane and compute	Controller nodes run the API services, schedulers, the database and the message queue.	Running a single controller in production.
What they all depend on	Two pieces of shared infrastructure sit under almost every service.	Expecting public-cloud features (such as managed databases) to exist without installing them.
What a first request looks like	A successful token issue proves that Keystone works and your credentials are right.	If that fails, nothing else will work.
Why choose it, and what it costs	A private cloud gives control and predictable cost at scale.	Skipping upgrades until the gap is too big.

Flow: The core services to Control plane and compute / Control plane and compute to What they all depend on / What they all depend on to What a first request looks like / What a first request looks like to Why choose it, and what it costs

Incident: Every service broke at once

Incident: *Nova, Neutron and Cinder all reported errors within the same minute.*

```
$ openstack compute service list
```

MARGA by EknathaLabs marga.eknathalabs.com

```
| nova-conductor | ctl-1 | internal | enabled | down |
| nova-scheduler | ctl-1 | internal | enabled | down |
$ sudo rabbitmqctl cluster_status | head -3
Error: unable to perform an operation on node 'rabbit@ctl-1'. Please see diagnostics information a \
    nd suggestion below.
```

The services were healthy; the message queue on the controller had stopped after its disk filled with logs. Services could not talk to each other, so each reported its own error. Freeing the disk and restarting RabbitMQ brought everything back within minutes. The team added disk alerts for the queue and treats a burst of unrelated failures as a sign to look at shared dependencies.

Common mistakes

- Treating each service as independent and ignoring the queue and database
- Running a single controller in production
- Expecting public-cloud features (such as managed databases) to exist without installing them
- Skipping upgrades until the gap is too big

Check yourself

Which OpenStack service creates virtual machines?

Answer: Nova

Which service is closest to AWS IAM?

Answer: Keystone

Many OpenStack services fail at the same time. What do you check first?

Answer: The message queue and the database

Where do virtual machines actually run?

Answer: On compute nodes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Identity, projects and quotas

13 minutes. Understand users, projects, roles and tokens, how quotas limit a project, and how to read an authentication failure.

Before anything is created in OpenStack, someone has to prove who they are and say which project the work belongs to. Keystone handles that.

What you will learn

- Domains, projects, users and roles
- Tokens, the service catalog and application credentials
- Quotas and how a project hits them
- Reading common authentication errors

The model

- A **domain** groups users and projects, which lets one cloud serve several organisations.
- A **project** (older name: tenant) owns resources such as servers and networks, and quotas are applied to it.
- A **user** is a person or a service account.
- A **role** is a name such as `admin` or `member`. A user gets a role **on a project**. The same user can be admin on one project and a plain member on another.

Permissions are decided by policy files in each service, which map roles to actions. By default, members can manage resources in their project, and `admin` can manage the whole cloud.

Tokens and the catalog

You sign in and receive a **token** (a temporary credential), which you present to every service. The token is scoped to one project. The **service catalog** returned with it lists the API endpoints, so the client knows where Nova and Neutron live.

```
$ env | grep ^OS_
OS_AUTH_URL=https://cloud.example.com:5000/v3
OS_PROJECT_NAME=shop
OS_USERNAME=asha
OS_USER_DOMAIN_NAME=Default
OS_PROJECT_DOMAIN_NAME=Default
$ openstack catalog list -c Name -c Type
+----------+----------+
| Name     | Type     |
+----------+----------+
| keystone | identity |
| nova     | compute  |
| neutron  | network  |
+----------+----------+
```

For automation, prefer **application credentials**: a secret limited to one project and, if you choose, to certain roles, that can be revoked without changing a person's password.

Quotas

Each project has limits on instances, vCPUs, RAM, volumes, floating IPs, security groups and more. They stop one team from using the whole cloud.

```
$ openstack quota show -c cores -c instances -c ram
+-----------+-------+
| Resource  | Limit |
+-----------+-------+
| cores     | 20    |
| instances | 10    |
| ram       | 51200 |
+-----------+-------+
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ openstack server create --flavor m1.xlarge --image ubuntu app-9
Quota exceeded for cores: Requested 8, but already used 16 of 20 cores (HTTP 403)
```

A quota error is not a fault in the cloud: it is a rule. The remedies are to delete unused resources, use a smaller flavor, or ask an admin to raise the quota. Raising quotas for everyone to avoid the question removes the protection.

Reading authentication errors

- **HTTP 401 Unauthorized**: the token or password is wrong or expired. Check the `OS_` variables and the domain names. A password right for one domain fails in another.
- **HTTP 403 Forbidden**: you are authenticated but the policy denies the action for your role. Ask for the right role on the project.
- **Could not find project**: the project name is wrong, or the project is in a different domain.
- **Clock skew**: tokens carry timestamps. A server whose clock is far off will reject valid tokens. Keep NTP running on every node.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Identity, projects and quotas fits together

Part	What it does	What goes wrong
The model	A domain groups users and projects, which lets one cloud serve several organisations.	Using the admin account for everyday work.
Tokens and the catalog	You sign in and receive a token (a temporary credential), which you present to every service.	Forgetting that names are per domain.
Quotas	Each project has limits on instances, vCPUs, RAM, volumes, floating IPs, security groups and more.	Giving the admin role to make an error go away.
Reading authentication errors	HTTP 401 Unauthorized: the token or password is wrong or expired.	A password right for one domain fails in another.

Flow: The model to Tokens and the catalog / Tokens and the catalog to Quotas / Quotas to Reading authentication errors

Incident: Everything worked yesterday

Incident: *A deployment script began failing at 08:00 with 401 errors.*

```
$ openstack token issue
The request you have made requires authentication. (HTTP 401)
$ openstack --os-cloud ci user show ci-deployer -c enabled -c password_expires_at
+---------------------+---------------------+
| Field               | Value               |
+---------------------+---------------------+
| enabled             | True                |
| password_expires_at | 2026-11-03T00:00:00 |
+---------------------+---------------------+
```

A password policy set 90-day expiry on all users, including the service account used by the pipeline. It had expired at midnight. The team moved the pipeline to an application credential, which they rotate on a schedule they choose, and exempted service users from password expiry rules. Automation should not share a person's login lifecycle.

Common mistakes

- Using the admin account for everyday work

- Giving the `admin` role to make an error go away
- Sharing one user between pipelines
- Forgetting that names are per domain

Check yourself

What owns resources and has quotas in OpenStack?

Answer: A project

What does HTTP 403 Forbidden usually mean in OpenStack?

Answer: You are signed in but policy denies the action for your role

Why are application credentials better for automation?

Answer: They are limited to a project and can be revoked without changing a person's password

A server create fails with Quota exceeded for cores. What is the correct reading?

Answer: A project limit was reached, which is a rule rather than a fault

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Compute: flavors, images and scheduling

14 minutes. Understand how Nova picks a host for a server, and diagnose servers stuck in ERROR with "No valid host was found".

Booting a server looks simple from the outside: pick a flavor and an image, press create. Inside, Nova has to find a compute node with enough room and then build the machine there. When it cannot, the server ends in ERROR.

What you will learn

- Flavors, images and the boot sequence
- How the scheduler and Placement choose a host
- Server states and the fault field
- Overcommit ratios and host aggregates

Flavors and images

A **flavor** defines size: vCPUs, RAM and root disk. An **image** (from Glance) is the starting disk. A server is a flavor plus an image plus a network. Cloud images use **cloud-init** to set up users and keys on first boot, using metadata that Nova provides.

```
$ openstack flavor list -c Name -c RAM -c VCPUs -c Disk
+----------+-------+-------+------+
| Name     |   RAM | VCPUs | Disk |
+----------+-------+-------+------+
| m1.small |  2048 |     1 |   20 |
| m1.large |  8192 |     4 |   80 |
| m1.xlarge| 16384 |     8 |  160 |
+----------+-------+-------+------+
```

The boot sequence

1. The API checks the request and your quota, and stores the server in the database.
2. The **scheduler** asks **Placement** which compute nodes have enough free resources, then applies filters (for example, the right availability zone or aggregate) and weighs the candidates.
3. The chosen compute node downloads the image, asks Neutron for a port and Cinder for volumes if needed, and starts the guest through libvirt.
4. The state moves from `BUILD` to `ACTIVE`.

Any step can fail. The server then shows `ERROR` and a **fault** message.

```
$ openstack server show app-9 -c status -c fault
+--------+-------------------------------------------------------------------------------+
| Field  | Value                                                                         |
+--------+-------------------------------------------------------------------------------+
| status | ERROR                                                                         |
| fault  | {'code': 500, 'message': 'No valid host was found. There are not enough hosts available \
    .'} |
+--------+-------------------------------------------------------------------------------+
```

"No valid host was found"

This is the scheduler saying that no compute node passed every check. It does not say which check failed. Work through the common reasons:

- **Not enough free RAM, vCPU or disk** on any one node for this flavor. A flavor larger than the biggest free slot cannot be placed even if the total free across the cloud is large.
- **The node is disabled or down.** `openstack compute service list` shows each nova-compute as `up` or `down`.

MARGA by EknathaLabs marga.eknathalabs.com

- **Wrong availability zone or aggregate**, or a flavor that requires special hardware properties that no node has.
- **Affinity rules** that cannot be satisfied.
- **Volume or network placement** that limits where the server can go.

```
$ openstack compute service list --service nova-compute -c Host -c Status -c State
+---------+---------+-------+
| Host    | Status  | State |
+---------+---------+-------+
| cmp-1   | enabled | up    |
| cmp-2   | enabled | down  |
| cmp-3   | disabled| up    |
+---------+---------+-------+
$ openstack hypervisor stats show -c memory_mb_used -c memory_mb
```

The scheduler log (`nova-scheduler.log` on the controller) lists how many hosts each filter rejected. Reading it turns a vague error into a specific one.

Overcommit

By default OpenStack allows more vCPUs than physical cores (the allocation ratio for CPU is 16 to 1) and some RAM overcommit (the default ratio is 1.5 to 1 in current releases). Operators change these ratios per cluster. Aggregates group hosts with different settings, such as a "gold" group at 1 to 1 for databases.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Compute: flavors, images and scheduling fits together

Part	What it does	What goes wrong
Flavors and images	A flavor defines size: vCPUs, RAM and root disk.	Changing flavors that live servers use.
The boot sequence	The API checks the request and your quota, and stores the server in the database.	Reading "No valid host" as meaning the cloud is full.
"No valid host was found"	This is the scheduler saying that no compute node passed every check.	Leaving failed ERROR servers around: they still count against quota.
Overcommit	By default OpenStack allows more vCPUs than physical cores (the allocation ratio for CPU is 16 to 1) and some RAM overcommit (the default ratio is 1.5 to 1 in current...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Flavors and images to The boot sequence / The boot sequence to "No valid host was found" / "No valid host was found" to Overcommit

Incident: A big flavor and a half-empty cloud

Incident: *A team could not boot a 16 GB server even though the cloud showed 200 GB free.*

```
$ openstack server create --flavor m1.xlarge --image ubuntu db-1
$ openstack server show db-1 -c status -c fault -f value
ERROR
{'code': 500, 'message': 'No valid host was found. There are not enough hosts available.'}
$ openstack resource provider usage show cmp-1
MEMORY_MB used 61440 total 65536
```

Total free memory was spread as small gaps over many compute nodes: each node had under 8 GB free, so none could hold a 16 GB server. The cloud was fragmented, not full. The operators moved some servers to open up room on one node and, for the long term, reserved a set of large hosts in an aggregate for large flavors. Free space has to be free on one node.

Common mistakes

- Reading "No valid host" as meaning the cloud is full
- Not checking whether compute services are up
- Changing flavors that live servers use
- Leaving failed ERROR servers around: they still count against quota

Check yourself

What defines the vCPU, RAM and disk size of a server?

Answer: The flavor

What does No valid host was found mean?

Answer: No compute node passed all the scheduler checks

Which command shows whether nova-compute services are up?

Answer: openstack compute service list

The cloud has 200 GB free RAM in total, but a 16 GB server fails to place. Why?

Answer: The free memory is spread over many nodes, and none has 16 GB free

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Networking: Neutron, security groups and floating IPs

15 minutes. Follow a packet from a server to the internet, and fix a server you cannot reach with SSH.

Neutron gives each project its own virtual networks, routers and firewalls. Most problems reported as "the server is broken" turn out to be network problems.

What you will learn

- Networks, subnets, ports and routers
- Provider and self-service networks
- Security groups and floating IPs
- A checklist for "I cannot reach my server"

The building blocks

- A **network** is a virtual layer-2 segment, and a **subnet** is an address range on it, with DHCP.
- A **port** is a virtual network card. A server gets one port per interface.
- A **router** connects subnets and joins a private network to an external one.
- **Provider networks** map directly to a physical network, such as a VLAN, and are managed by the operators. **Self-service networks** (tenant networks) are created by projects, and use overlays (VXLAN or Geneve) so that many projects can use the same address ranges without clashing.
- The implementation behind it is Open vSwitch or OVN on each node, plus agents for DHCP, metadata and routing.

```
$ openstack network list -c Name -c Subnets
+----------+--------------------------------------+
| Name     | Subnets                              |
+----------+--------------------------------------+
| public   | 5a0c... (203.0.113.0/24)             |
| private  | 7d21... (10.0.0.0/24)                |
+----------+--------------------------------------+
```

Security groups

A **security group** is a set of allow rules applied to a server's port. The default group allows all outbound traffic and denies inbound traffic from outside, except from members of the same group. There are no deny rules: if nothing allows it, it is dropped. A port can have several groups, and their rules add up.

```
$ openstack security group rule list web --long -c "IP Protocol" -c "Port Range" -c "IP Range" -c  \
    Direction
+-------------+------------+-----------+-----------+
| IP Protocol | Port Range | IP Range  | Direction |
+-------------+------------+-----------+-----------+
| tcp         | 80:80      | 0.0.0.0/0 | ingress   |
+-------------+------------+-----------+-----------+
$ openstack security group rule create --proto tcp --dst-port 22 --remote-ip 203.0.113.7/32 web
```

Opening a port to `0.0.0.0/0` exposes it to the whole internet. Limit management ports such as SSH to known addresses or a jump host.

Floating IPs

Servers on private networks have private addresses. A **floating IP** is a public address that Neutron maps to a server's private address using NAT at the router. It can be moved to another server, which is useful for failover.

```
$ openstack floating ip create public -c floating_ip_address -f value
203.0.113.45
$ openstack server add floating ip web-1 203.0.113.45
```

MARGA by EknathaLabs marga.eknathalabs.com

"I cannot reach my server"

Check from the inside out:

1. Is the server `ACTIVE`, and did it boot? `openstack console log show web-1` shows the boot log and whether it got an address from DHCP.
2. Does the port have the right fixed IP? `openstack port list --server web-1`.
3. Does the security group allow your traffic from your address, on the right port and protocol?
4. Is there a floating IP, and is the router attached to the external network?
5. Is the service listening inside the server? Console or another server on the same network can test.
6. Does the server's own firewall (iptables, ufw) allow it?

A frequent cause is steps 3 and 4: either the rule is missing or the floating IP is not attached.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Networking: Neutron, security groups and floating... fits together

Part	What it does	What goes wrong
The building blocks	A network is a virtual layer-2 segment, and a subnet is an address range on it, with DHCP.	Assuming a floating IP exists because the server has a private address.
Security groups	A security group is a set of allow rules applied to a server's port.	Forgetting that security groups only allow, and default denies inbound.
Floating IPs	Servers on private networks have private addresses.	Changing the router when the security group is the cause.
"I cannot reach my server"	Check from the inside out.	"I cannot reach my server" Check from the inside out: 1.

Flow: The building blocks to Security groups / Security groups to Floating IPs / Floating IPs to "I cannot reach my server"

Incident: SSH works from the office but not from home

Incident: *An engineer working from home could not SSH to a server that colleagues in the office reached fine.*

```
$ openstack security group rule list bastion --long -c "Port Range" -c "IP Range" -c Direction
+------------+-----------------+-----------+
| Port Range | IP Range        | Direction |
+------------+-----------------+-----------+
| 22:22      | 198.51.100.0/24 | ingress   |
+------------+-----------------+-----------+
$ curl -s ifconfig.me
203.0.113.88
```

The security group allowed SSH only from the office network, 198.51.100.0/24. The engineer's home address, 203.0.113.88, was outside it. The fix was to add a rule for that single address with a /32, not to open SSH to the world, and later to move access behind a VPN. The cloud was behaving exactly as configured.

Common mistakes

- Opening 22 or 3389 to 0.0.0.0/0 to make a problem go away
- Forgetting that security groups only allow, and default denies inbound
- Assuming a floating IP exists because the server has a private address
- Changing the router when the security group is the cause

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does a security group contain?

Answer: Allow rules only; anything not allowed is dropped

What is a floating IP?

Answer: A public address that Neutron maps to a server's private address

SSH works from the office but not from home. Which is the likely cause?

Answer: The security group allows SSH only from the office range

What do self-service networks use to keep projects apart?

Answer: Overlay tunnels such as VXLAN or Geneve

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Operating OpenStack: logs, upgrades and troubleshooting

15 minutes. Find the right log fast, recover stuck servers and volumes, and plan upgrades and capacity for a private cloud.

Running a private cloud is an operations job. Most days it is routine, and the skill is knowing where to look when it is not.

What you will learn

- Where each service logs and how to follow a request
- Stuck states and safe ways to reset them
- Health checks for the shared dependencies
- Upgrades, backups and capacity

Following a request

Every API request has a **request id** that appears in the logs of each service it touches. When a user reports a failed server creation, get the request id and search for it.

```
$ openstack --debug server create --flavor m1.small --image ubuntu t1 2>&1 | grep -i "x-openstack- \
    request-id" | head -1
x-openstack-request-id: req-6b9f2a5c-1d1e-4a63-a1f0-2d9a8cb7e411
$ sudo grep req-6b9f2a5c /var/log/nova/*.log | cut -c1-140
nova-api.log:  req-6b9f... POST /v2.1/servers  status: 202
nova-conductor.log: req-6b9f... Failed to schedule instances: NoValidHost_Remote
nova-scheduler.log: req-6b9f... Filter ComputeFilter returned 0 hosts
```

The trail leads from the API to the scheduler, which names the filter that rejected every host. Log paths vary by deployment: `/var/log/nova/` on package installs, `/var/log/kolla/nova/` or container logs for Kolla-based ones. Each service has its own: nova-api, nova-scheduler, nova-conductor, nova-compute, neutron-server, cinder-volume and so on.

Health of the shared parts

```
$ openstack compute service list -c Binary -c Host -c State
$ openstack network agent list -c "Agent Type" -c Host -c Alive
$ openstack volume service list -c Binary -c Host -c State
$ sudo rabbitmqctl list_queues name messages | sort -k2 -n -r | head -3
$ mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
```

A down agent means that host's part of the service is gone. A queue with thousands of waiting messages means consumers are not keeping up or have stopped. A Galera database that has lost members may have lost quorum and refuse writes.

Stuck states

Servers can stay in `BUILD`, `DELETING` or `ERROR`. Volumes can stay in `attaching` or `error_deleting`. Usually the cause is that a message was lost or an agent was down at the time. Steps:

1. Read the fault and the logs: do not reset blindly.
2. Check that the target compute or storage service is healthy now.
3. For servers, `openstack server set --state active <id>` (admin) corrects the database record, but does not start anything. For volumes, `openstack volume set --state available <id>`.
4. Then retry the real operation, and confirm in the hypervisor (`virsh list`) that the truth matches the record.

Resetting state only edits the database. If the guest is really running or the volume is really attached, a wrong reset creates a second copy or corrupts data. Compare reality first.

MARGA by EknathaLabs marga.eknathalabs.com

Upgrades and backups

OpenStack releases about every six months. Upgrading means moving service by service, with database migrations, usually from one release to the next (some releases support skipping). Practise in a test cloud with a copy of the real data. Always:

- Back up the databases and configuration before every step, and prove a restore works.
- Read the release notes for removed settings.
- Upgrade the control plane before compute nodes, which can run older agents for a short time.
- Keep the plan reversible where possible.

Capacity

Watch the ratio of used to allocatable vCPU, RAM and storage, with headroom to lose a compute node and still re-run its servers. Track the number of ERROR servers, failed API requests, queue depth and database size. A cloud that is at 90 percent in one flavor class will fail large requests even when totals look fine.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Operating OpenStack: logs, upgrades and... fits together

Part	What it does	What goes wrong
Following a request	Every API request has a request id that appears in the logs of each service it touches.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Health of the shared parts	A down agent means that host's part of the service is gone.	No alert on queue depth or Galera cluster size.
Stuck states	Servers can stay in BUILD, DELETING or ERROR.	Resetting states without checking the hypervisor.
Upgrades and backups	OpenStack releases about every six months.	Upgrading without a tested restore of the database.
Capacity	Watch the ratio of used to allocatable vCPU, RAM and storage, with headroom to lose a compute node and still re-run its servers.	Treating a three-controller cluster as safe while one node is down and unnoticed.

Flow: Following a request to Health of the shared parts / Health of the shared parts to Stuck states / Stuck states to Upgrades and backups / Upgrades and backups to Capacity

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The volume that would not detach

Incident: *A volume stuck in* `detaching` *blocked a database migration.*

```
$ openstack volume show data-vol -c status -c attachments
| status      | detaching |
| attachments | [{'server_id': '3f2a', 'device': '/dev/vdb'}] |
$ sudo virsh domblklist instance-0000004a
 vdb   /dev/rbd/volumes/volume-91c2
$ openstack compute service list --service nova-compute -c Host -c State | grep cmp-7
| cmp-7 | down |
```

The compute node cmp-7 was down, so nothing could complete the detach. The operators first confirmed that the server was not running on any other host, so it was safe, then restored cmp-7. After it came up, they retried the detach. Had they only reset the volume to `available` while the guest still had it attached, two writers could have corrupted the data. They now have a runbook that starts with "check the compute service, then compare the hypervisor with the database".

Common mistakes

- Resetting states without checking the hypervisor

- Upgrading without a tested restore of the database
- No alert on queue depth or Galera cluster size
- Treating a three-controller cluster as safe while one node is down and unnoticed

Check yourself

What helps you follow one failed request across services?

Answer: The request id in the logs

What does openstack server set --state active do?

Answer: Edits the database record; it does not start anything

Why compare the hypervisor with the database before resetting a state?

Answer: Resetting a state while the guest is really attached or running can cause duplicate writers

What should be done before every upgrade step?

Answer: Back up the databases and prove a restore

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. OpenStack operations: "No valid host" and scheduling failures

13 minutes. Trace a failed server build from the API through the scheduler and Placement, read the real reason for No valid host found, and fix quota, capacity and aggregate mismatches.

"No valid host was found" is the most common OpenStack build error, and it says almost nothing by itself. It means that no compute node passed every filter for this request. The skill is to find which filter rejected the hosts and why.

What you will learn

- The path from boot request to compute node
- How Placement and the scheduler pick hosts
- Reading the real reason in the logs and instance faults
- Typical causes: capacity, flavor, aggregates, availability zones and cells

The path of a request

`nova-api` accepts the request and stores the instance. `nova-conductor` coordinates. The `nova-scheduler` asks **Placement** for candidate resource providers that have enough VCPU, memory and disk (and any traits or resources the flavor requires), then applies filters (availability zone, aggregate matching, image properties, anti-affinity) and weighs the survivors. The chosen compute node builds the guest, asking Neutron for ports and Cinder for volumes. A failure at any step sends the instance to `ERROR`.

```
$ openstack server show web-01 -c status -c fault
$ openstack server event list web-01
$ openstack flavor show m1.large
$ openstack compute service list
$ openstack hypervisor list
```

The `fault` field and the event list tell you the request ID, which you can search in the scheduler and conductor logs.

Ask Placement

```
$ openstack resource provider list
$ openstack resource provider usage show 7c6a1c2e-1111-4a8b-9ef2-0d7d5d5a0a11
$ openstack resource provider inventory list 7c6a1c2e-1111-4a8b-9ef2-0d7d5d5a0a11
$ openstack allocation candidate list --resource VCPU=8 --resource MEMORY_MB=32768 --resource DISK \
    _GB=100
```

The candidate list answers: for this size, which providers fit? An empty list means a real capacity problem (including the allocation ratios) or a request that asks for something no node offers (a required trait, a very large flavor, PCI devices). If candidates exist but the build still fails, a scheduler filter rejected them: read the scheduler log with the request ID.

Common causes

- **No capacity** at the configured allocation ratios, or hosts disabled or down (`compute service list` shows `State: down` or `Status: disabled`).
- **Flavor asks for too much** (extra specs for NUMA, huge pages, a trait) and few hosts match.
- **Aggregate or AZ mismatch**: the image or flavor targets an aggregate with no free hosts, or the AZ has none.
- **Anti-affinity**: a server group forbids placing on the few remaining hosts.
- **New hosts not mapped**: compute services exist, but `nova-manage cell_v2 discover_hosts` was not run, so the cell does not know them.
- **Quota** errors are different: they are rejected before scheduling, with a message about limits.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How OpenStack operations: "No valid host" and... fits together

Part	What it does	What goes wrong
The path of a request	nova-api accepts the request and stores the instance.	Not following the request ID across services.
Ask Placement	The candidate list answers: for this size, which providers fit?	Reading No valid host as a pure capacity problem.
Common causes	No capacity at the configured allocation ratios, or hosts disabled or down (compute service list shows State: down or Status: disabled).	Showing total capacity without breaking it down by flavor class.

Flow: The path of a request to Ask Placement / Ask Placement to Common causes

Incident: Plenty of room, nowhere to put it

Incident: *A project could not create instances of its new flavor, while the dashboard showed 40 percent free capacity.*

```
$ openstack flavor show m1.numa -c properties
properties | hw:cpu_policy='dedicated', hw:mem_page_size='large'
$ openstack allocation candidate list --resource VCPU=8 --resource MEMORY_MB=32768 --resource DISK \
    _GB=50 | wc -l
0
$ openstack hypervisor list --long | head -4
```

The flavor needed dedicated cores and huge pages. Only two hosts in a small aggregate were configured for that, and both were full, while the free capacity was on general-purpose hosts. The operations team added hosts to the aggregate with the right configuration, made the dashboard show capacity per aggregate and per flavor class, and documented how special flavors map to host pools.

Common mistakes

- Reading No valid host as a pure capacity problem
- Not following the request ID across services
- Forgetting host discovery after adding compute nodes
- Showing total capacity without breaking it down by flavor class

Check yourself

What does Placement provide?

Answer: An inventory and allocation service that finds hosts with enough resources

Which command shows whether any provider fits a requested size?

Answer: openstack allocation candidate list with resource requests

After adding a new compute node, instances do not land on it. A likely missing step?

Answer: Host discovery in the cell with nova-manage cell_v2 discover_hosts

How do you follow one failed build across services?

Answer: Search logs for the request ID from the instance fault or event

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. OpenStack operations: Neutron and Ceph troubleshooting

13 minutes. Trace a missing network path through agents, bridges and namespaces, and read Ceph health to understand slow or stuck volumes and degraded placement groups.

Two subsystems cause most deep OpenStack incidents: virtual networking, where a packet crosses several software devices, and storage, usually Ceph, where cluster health decides whether volumes respond. Each can be traced step by step.

What you will learn

- The path of a packet from a guest to the outside
- Checking agents, ports and flows with the standard tools
- Reading `ceph -s` and health details
- Fixes for slow requests, full OSDs and degraded data

Neutron: follow the packet

First check the control plane: are the agents alive, and is the port bound and active?

```
$ openstack network agent list
$ openstack port list --server web-01
$ openstack port show 3f1a2b4c-0000-4d5e-9aaa-111122223333 -c status -c binding_vif_type -c bindin \
    g_host_id
$ openstack floating ip list
$ openstack security group rule list web-sg
```

A port with status `DOWN` or a binding failure points to the agent or driver on that host. Missing security group rules are a common cause of "can't connect" even when everything else is fine. Then trace the data path on the compute host (shown for Open vSwitch):

```
$ sudo ovs-vsctl show
$ sudo ovs-ofctl dump-flows br-int | head
$ sudo ip netns list
$ sudo ip netns exec qrouter-0a1b2c3d ip addr
$ sudo tcpdump -i tap3f1a2b4c-00 -nn icmp
```

Capture at each hop: the guest's tap device, the bridge, the tunnel or VLAN interface, and the router or DHCP namespace on the network node. The first hop where packets appear but then vanish tells you which rule, MTU or link to fix. Tunnels (VXLAN or Geneve) reduce the effective MTU, so large packets failing while small ones work usually means an MTU mismatch between guest, tunnel and physical network.

Ceph: read the health

```
$ ceph -s
$ ceph health detail
$ ceph osd tree
$ ceph osd df
$ ceph pg stat
$ ceph osd perf
```

`ceph -s` shows overall health, monitor quorum, OSDs up and in, and placement group (PG) states. `HEALTH_OK` is the goal. `HEALTH_WARN` with `degraded` or `misplaced` PGs means data is being recovered; the cluster serves I/O but with less protection and extra load. Watch for:

- **Slow ops** and a high `commit_latency` in `osd perf`: a failing disk, a saturated network or a recovery that is competing with client traffic.
- **Near full OSDs**: Ceph stops writes when OSDs reach the full ratio. Add capacity, rebalance, or delete data before it happens; never ignore the nearfull warning.

MARGA by EknathaLabs
marga.eknathalabs.com

- **OSDs down**: check the host, the disk and the service. A single failed disk is routine; many at once suggests network or power.

Recovery speed can be tuned (`osd_max_backfills` and recovery settings) to balance speed against client impact, which is useful during business hours.

From the OpenStack side

Cinder volumes and Nova ephemeral disks backed by RBD are slow or stuck when the Ceph pool is unhealthy. If an instance freezes on I/O, check Ceph health before restarting anything, and use `rbd` to inspect images and locks:

```
$ rbd ls volumes | head
$ rbd status volumes/volume-0a1b2c3d
$ rbd info volumes/volume-0a1b2c3d
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How OpenStack operations: Neutron and Ceph... fits together

Part	What it does	What goes wrong
Neutron: follow the packet	First check the control plane: are the agents alive, and is the port bound and active?	Capturing packets at only one point.
Ceph: read the health	ceph -s shows overall health, monitor quorum, OSDs up and in, and placement group (PG) states.	Restarting instances before checking Ceph health.
From the OpenStack side	Cinder volumes and Nova ephemeral disks backed by RBD are slow or stuck when the Ceph pool is unhealthy.	Forgetting overlay overhead when setting MTUs.

Flow: Neutron: follow the packet to Ceph: read the health / Ceph: read the health to From the OpenStack side

Incident: The cluster that stopped accepting writes

Incident: *At 14:00 many instances froze, and Cinder operations timed out.*

```
$ ceph health detail
HEALTH_ERR 3 full osd(s); 1 pool(s) full
osd.12 is full at 95%
$ ceph osd df | sort -k17 -nr | head -3
12  hdd  ... 96.20
```

Several OSDs on one host had filled up faster than the others because of an uneven placement and a large batch import. At the full ratio Ceph blocks writes to protect data. The operators temporarily raised the full ratio a little, deleted old snapshots to free space, rebalanced with the balancer module, and then added disks. They created alerts at 70 and 80 percent and a capacity forecast, so nearfull would never again be a surprise.

Common mistakes

- Restarting instances before checking Ceph health
- Ignoring nearfull warnings
- Capturing packets at only one point
- Forgetting overlay overhead when setting MTUs

Check yourself

What does an agent in the DOWN state suggest?

Answer: The service on that host is not reporting, so ports there may not be wired

MARGA by EknathaLabs marga.eknathalabs.com

What happens when Ceph OSDs reach the full ratio?

Answer: Writes are blocked to protect the data

Which command gives a quick overall view of Ceph?

Answer: ceph -s

Why capture at several points in the virtual network path?

Answer: To find the hop where packets stop appearing

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. OpenStack operations: maintenance, evacuation and a day-two runbook

12 minutes. Take compute hosts out of service safely with live migration and evacuation, handle host failures, and keep a runbook with checks for upgrades and capacity.

A private cloud has hundreds of moving parts, and hardware fails every week at scale. The difference between a calm operations team and a stressed one is a short set of practiced procedures: drain a host, recover from a dead host, patch without downtime and check health before and after any change.

What you will learn

- Draining a compute host for planned maintenance
- Evacuating instances from a failed host
- Pre-change and post-change health checks
- What belongs in a day-two runbook

Planned maintenance: drain

1. Check that other hosts in the aggregate have capacity (N+1).
2. Stop new placements on the host by disabling its compute service.
3. Live migrate instances away (or cold migrate those that cannot move live).
4. Do the work, reboot and verify.
5. Enable the service again and watch placement.

```
$ openstack compute service set --disable --disable-reason "kernel patch 2026-03-05" compute-12 no \
    va-compute
$ openstack server list --all-projects --host compute-12 -c ID -c Name -c Status
$ openstack server migrate --live-migration --block-migration 5d2c7e1a-0000-4c11-9d0a-3f4e8a1b2c3d
$ openstack server migration list --status running
$ openstack compute service set --enable compute-12 nova-compute
```

Live migration fails when the destination has an incompatible CPU model, a different network or storage setup, or when the guest changes memory too quickly. Migrate in small batches, check each one finishes, and keep an eye on the network. Instances using local storage need block migration, which is slower. Pinned or SR-IOV guests may require cold migration with a short downtime that you plan with their owners.

Host failure: evacuate

If a host dies, instances on it are stuck in their last state. Make sure the host is really down and **fenced** (powered off or isolated), because rebuilding an instance elsewhere while the old one still runs against shared storage can corrupt data.

```
$ openstack compute service list --service nova-compute
$ openstack server list --host compute-12 --all-projects
$ openstack server evacuate 5d2c7e1a-0000-4c11-9d0a-3f4e8a1b2c3d
```

Evacuation rebuilds the instance on another host using shared or boot-from-volume storage. Instances with local ephemeral disks lose that data, which is a good reason to prefer boot from volume for anything important. Some deployments automate this with a high-availability service; test it regularly, with clear rules about when it triggers.

Health checks around every change

Before: all compute services up, all network agents alive, Ceph `HEALTH_OK`, no running migrations, no active alerts, recent backups of the database and configuration. After: the same checks, plus a canary test that creates a server, attaches a volume, assigns a floating IP, pings it and deletes it. Automate the canary and run it continuously.

MARGA by EknathaLabs marga.eknathalabs.com

```
#!/usr/bin/env bash
set -euo pipefail
openstack compute service list --service nova-compute -f value -c State | grep -vq down
openstack network agent list -f value -c Alive | grep -vq False
ceph health | grep -q HEALTH_OK
echo "preflight ok"
```

The day-two runbook

Keep it short, versioned and tested: contact and escalation list, architecture map, how to drain and evacuate, how to restart each service safely, how to find logs, upgrade steps with rollback points, capacity thresholds and who approves changes. Review after every incident, and link each alert to its runbook section.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How OpenStack operations: maintenance, evacuation and... fits together

Part	What it does	What goes wrong
Planned maintenance: drain	Check that other hosts in the aggregate have capacity (N+1).	Draining a host without checking spare capacity.
Host failure: evacuate	If a host dies, instances on it are stuck in their last state.	Using local ephemeral disks for important data.
Health checks around every change	Before: all compute services up, all network agents alive, Ceph HEALTH_OK, no running migrations, no active alerts, recent backups of the database and configuration.	Skipping the canary test after a change.
The day-two runbook	Keep it short, versioned and tested: contact and escalation list, architecture map, how to drain and evacuate, how to restart each service safely, how to find logs...	Evacuating without fencing the failed host.

Flow: Planned maintenance: drain to Host failure: evacuate / Host failure: evacuate to Health checks around every change / Health checks around every change to The day-two runbook

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The evacuation that created duplicates

> **Incident:** *A compute host lost its network but kept running, an operator evacuated its instances, and several databases wrote to the same volumes from two hosts.*

```
$ openstack compute service list --service nova-compute | grep compute-12
| compute-12 | nova-compute | nova | enabled | down |
$ ping -c 2 compute-12-ipmi
2 packets transmitted, 2 received
```

The service showed down, but the host was alive and still running its guests. The evacuation started new copies elsewhere while the originals were still running, and two instances held the same volume. The team recovered from backups and wrote a rule: confirm power state through the out-of-band controller and power the host off (fencing) before any evacuation. They later automated that check inside the evacuation tool.

Common mistakes

- Evacuating without fencing the failed host
- Draining a host without checking spare capacity
- Using local ephemeral disks for important data
- Skipping the canary test after a change

Check yourself

What should you do before migrating instances off a host?

Answer: Disable the compute service for new placements and check capacity elsewhere

Why fence a failed host before evacuation?

Answer: To prevent two copies of an instance from writing to the same storage

Why prefer boot from volume?

Answer: Instances can be rebuilt on another host with their data intact

What does a canary test verify after a change?

Answer: That servers, volumes and networking work end to end

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Cinder, Glance and OpenStack storage

12 minutes. Understand block volumes with Cinder, images with Glance and how they connect to Ceph, and fix common volume and image problems.

Compute gets the attention, but most day-to-day OpenStack problems involve storage: volumes that will not attach, images that are slow to boot and snapshots that fill pools.

What you will learn

- Cinder volumes, types, snapshots and backups
- Glance images, properties and formats
- How Ceph connects both services
- Troubleshooting attach and boot problems

Cinder

```
openstack volume type list
openstack volume create --size 100 --type ssd db-data
openstack server add volume db-1 db-data
openstack volume snapshot create --volume db-data db-data-before-upgrade
openstack volume backup create --name db-backup-1 db-data
```

A **volume type** maps to a backend and its settings, such as performance tier. A **snapshot** is a point-in-time copy on the same backend and is not a backup. A **backup** is stored in a separate service such as object storage.

Glance

```
openstack image create --file jammy.qcow2 --disk-format qcow2 --container-format bare \
  --property hw_scsi_model=virtio-scsi --property os_type=linux --public jammy-2026-11
openstack image list --status active
```

Image **properties** control hardware and scheduling, such as disk bus and required features. With Ceph, store images as **raw** so they can be cloned without conversion.

With Ceph

When Glance, Cinder and Nova all use Ceph RBD, creating a volume from an image is a copy-on-write clone in the same cluster, and instances can boot from volume without copying data over the network.

Troubleshooting

```
openstack volume show db-data -c status -c attachments
cinder-manage volume list 2>/dev/null | head
grep -i error /var/log/cinder/cinder-volume.log | tail
rbd -p volumes ls | head
```

Typical states: **error_attaching** (compute or iSCSI or RBD access problem), **stuck in deleting** (a snapshot still exists), **creating** for a long time (backend full or the scheduler found no valid backend). Check quotas, backend capacity and the volume service logs, and reset states only after confirming the real state on the backend.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Cinder, Glance and OpenStack storage fits together

Part	What it does	What goes wrong
Cinder	A volume type maps to a backend and its settings, such as performance tier.	Resetting volume states without checking the backend.
Glance	Image properties control hardware and scheduling, such as disk bus and required features.	Converting images on every boot.
With Ceph	When Glance, Cinder and Nova all use Ceph RBD, creating a volume from an image is a copy-on-write clone in the same cluster, and instances can boot from volume without...	With Ceph When Glance, Cinder and Nova all use Ceph RBD, creating a volume from an image is a copy-on-write clone in the same cluster, and...
Troubleshooting	Typical states: error_attaching (compute or iSCSI or RBD access problem), stuck in deleting (a snapshot still exists), creating for a long time (backend full or the...	Ignoring pool capacity.

Flow: Cinder to Glance / Glance to With Ceph / With Ceph to Troubleshooting

Incident: Volume stuck in error_attaching

Incident: *A database volume would not attach after a compute host replacement.*

```
status: error_attaching
libvirt: failed to open rbd volumes/volume-9c1f: Permission denied
ceph client.nova lacks access to pool volumes
```

The new host had a different Ceph key that lacked permission on the volumes pool. Once the capabilities were corrected and the secret defined on the host, the attach worked. The team added the Ceph credentials to the host build checks.

Common mistakes

- Treating snapshots as backups
- Resetting volume states without checking the backend
- Converting images on every boot
- Ignoring pool capacity

Check yourself

What does Cinder provide?

Answer: Block volumes

What does Glance store?

Answer: Images

Why use raw images with Ceph?

Answer: Fast copy-on-write clones

What is a snapshot not?

Answer: A backup in another failure domain

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 10. Horizon, Heat and OpenStack automation

12 minutes. Automate OpenStack with the CLI, SDKs, Heat templates and Terraform, and use Horizon for what it is good at.

OpenStack exposes every capability through REST APIs. The dashboard, command line, SDKs and orchestration tools are different clients of the same interface, and choosing the right one makes work repeatable.

What you will learn

- CLI and SDK basics with application credentials
- Heat templates and stacks
- Terraform with the OpenStack provider
- When the dashboard is the right tool

CLI and credentials

```
export OS_CLOUD=prod            # reads clouds.yaml
openstack server list --all-projects --status ERROR
openstack application credential create ci-deploy --role member
```

Use **application credentials** for automation instead of a user password. They can be limited and revoked without changing the person's account.

Python SDK

```
import openstack
conn = openstack.connect(cloud="prod")
for s in conn.compute.servers(status="ERROR"):
    print(s.name, s.fault["message"] if s.fault else "")
```

Heat

```
heat_template_version: 2021-04-16
parameters:
  image: {type: string, default: jammy-2026-11}
  flavor: {type: string, default: m1.medium}
resources:
  net:
    type: OS::Neutron::Net
  subnet:
    type: OS::Neutron::Subnet
    properties: {network: {get_resource: net}, cidr: 10.50.0.0/24}
  web:
    type: OS::Nova::Server
    properties:
      image: {get_param: image}
      flavor: {get_param: flavor}
      networks: [{network: {get_resource: net}}]
outputs:
  web_ip:
    value: {get_attr: [web, first_address]}
```

```
openstack stack create -t web.yaml demo
openstack stack event list demo
openstack stack delete demo --yes --wait
```

A **stack** groups resources that are created and deleted together. Heat tracks dependencies and rolls back on failure. Terraform offers similar declarative control with a shared workflow across clouds.

Dashboard

Horizon is useful for exploration, quotas and showing topology to new users. Do not rely on it for repeatable changes.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Horizon, Heat and OpenStack automation fits together

Part	What it does	What goes wrong
CLI and credentials	Use application credentials for automation instead of a user password.	Using personal passwords in automation.
Python SDK	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Heat	A stack groups resources that are created and deleted together.	Skipping stack events when debugging.
Dashboard	Horizon is useful for exploration, quotas and showing topology to new users.	Do not rely on it for repeatable changes.

Flow: CLI and credentials to Python SDK / Python SDK to Heat / Heat to Dashboard

Incident: The lab nobody could rebuild

Incident: *A hand-built environment was lost with its documentation.*

```
project: qa-lab
14 servers, 3 networks, 6 volumes
created by manual clicks, stack: none
```

A quota cleanup removed some of the lab's resources and no one remembered how they were configured. After rebuilding it as a Heat stack, the team could recreate the environment in minutes and review changes as code.

Common mistakes

- Hand-built environments
- Using personal passwords in automation
- Skipping stack events when debugging
- Leaving test stacks running forever

Check yourself

What does Heat use to describe resources?

Answer: Templates

Why use application credentials?

Answer: Limited and revocable automation access

What is a stack?

Answer: A group of resources managed together

When is Horizon best?

Answer: Exploration and visibility

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Deploying OpenStack with Kolla-Ansible

13 minutes. See how containerized OpenStack is deployed with Kolla-Ansible, configure inventory and globals, and run upgrades and reconfiguration safely.

Installing OpenStack service by service by hand is slow and error prone. Kolla packages each service as a container, and Kolla-Ansible deploys and manages them across your servers.

What you will learn

- The inventory and globals files
- The deployment workflow and prechecks
- Overriding service configuration
- Reconfigure and upgrade

Files

```
# multinode inventory (excerpt)
[control]
ctl-01
ctl-02
ctl-03
[network]
ctl-01
ctl-02
ctl-03
[compute]
cmp-01
cmp-02
[storage]
sto-01
```

```
# /etc/kolla/globals.yml (excerpt)
kolla_base_distro: ubuntu
openstack_release: "2025.1"
network_interface: eth0
neutron_external_interface: eth1
kolla_internal_vip_address: 10.20.0.250
enable_cinder: "yes"
enable_haproxy: "yes"
```

`globals.yml` holds cluster-wide choices. `passwords.yml` holds generated secrets and must be protected and backed up. Both belong in a private repository, with secrets encrypted.

Workflow

```
kolla-genpwd
kolla-ansible -i multinode bootstrap-servers
kolla-ansible -i multinode prechecks
kolla-ansible -i multinode deploy
kolla-ansible -i multinode post-deploy
```

Prechecks catch missing interfaces, wrong addresses, port conflicts and missing packages before anything changes. Fix every failure first.

Override configuration

Place files under `/etc/kolla/config/<service>/` to add or override service settings, for example `nova.conf`. Run `kolla-ansible reconfigure` to apply.

```
kolla-ansible -i multinode reconfigure --tags nova
docker ps --format 'table {{.Names}}\t{{.Status}}' | head
```

MARGA by EknathaLabs
marga.eknathalabs.com

Upgrade

Upgrade one release at a time, read the release notes, back up databases and configuration, run `pull` to fetch images ahead of time and then `upgrade`. Test the procedure in a staging cloud that mirrors production.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Deploying OpenStack with Kolla-Ansible fits together

Part	What it does	What goes wrong
Files	globals.yml holds cluster-wide choices. passwords.yml holds generated secrets and must be protected and backed up.	Losing passwords.yml.
Workflow	Prechecks catch missing interfaces, wrong addresses, port conflicts and missing packages before anything changes.	Skipping prechecks.
Override configuration	Place files under /etc/kolla/config/<service>/ to add or override service settings, for example nova.conf.	Hand-editing files inside containers.
Upgrade	Upgrade one release at a time, read the release notes, back up databases and configuration, run pull to fetch images ahead of time and then upgrade.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Files to Workflow / Workflow to Override configuration / Override configuration to Upgrade

Incident: A precheck that saved the day

Incident: *Prechecks found a wrong interface name before a deploy.*

```
TASK [neutron : Checking neutron_external_interface]
fatal: [ctl-02]: FAILED! interface eth1 not found
PLAY RECAP: ctl-02 failed=1
```

One controller had a different network card name. The precheck failed before any change was made. The team fixed the name in the inventory host variables, added a hardware check to server provisioning and reran the deploy successfully.

Common mistakes

- Skipping prechecks
- Losing passwords.yml
- Hand-editing files inside containers
- Upgrading several releases at once

Check yourself

What does globals.yml hold?

Answer: Cluster-wide deployment settings

What do prechecks do?

Answer: Find problems before deploying

Where do per-service overrides go?

Answer: /etc/kolla/config/<service>/

How upgrade releases?

Answer: One release at a time with backups

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. OpenStack monitoring and capacity planning

12 minutes. Monitor the control plane and compute capacity, alert on API and message queue health and plan quotas and overcommit with real data.

OpenStack has many services with many failure modes, and users only see the symptoms: slow API calls, instances that never become active and volumes that hang. A small set of signals covers most of them.

What you will learn

- Control plane signals: APIs, queue, database and Ceph
- Compute capacity and overcommit metrics
- Alerts that predict user-visible failure
- Forecasting capacity and setting quotas

Control plane

- **API health:** request latency and 5xx rates per service, through HAProxy or exporters.
- **Message queue:** RabbitMQ queue depth, unacked messages and consumers. Growing queues mean a stuck service.
- **Database:** connections, slow queries and replication state of Galera.
- **Service heartbeats:** `openstack compute service list` and `network agent list` show down services.
- **Ceph:** health status, near-full pools and slow operations.

```
openstack compute service list --long | grep -v " up "
openstack network agent list | grep -v ":-)"
rabbitmqctl list_queues name messages consumers | sort -k2 -nr | head
ceph -s
```

Capacity

```
openstack hypervisor stats show
openstack hypervisor list --long -c "Hypervisor Hostname" -c "vCPUs Used" -c "vCPUs" -c "Memory MB \
     Used" -c "Memory MB"
```

Track vCPU and memory used against capacity per host aggregate, the **overcommit ratio** and the number of instances of each flavor that could still be scheduled. Also track storage pools, floating IPs and network ports, because they run out too.

Alerts

Alert on service down for more than a few minutes, RabbitMQ queues rising, API error rates, Ceph warnings, compute hosts above a utilization threshold and failed instance builds. Keep a dashboard of **time to fit** the standard flavors.

Plan

Use growth trends and upcoming projects to forecast when capacity runs out, and account for procurement lead time. Set project quotas so that the sum is consistent with the overcommit you accept, and review them regularly.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How OpenStack monitoring and capacity planning fits together

Part	What it does	What goes wrong
Control plane	API health: request latency and 5xx rates per service, through HAProxy or exporters.	No alert on queue depth.
Capacity	Track vCPU and memory used against capacity per host aggregate, the overcommit ratio and the number of instances of each flavor that could still be scheduled.	Ignoring storage and network capacity.
Alerts	Alert on service down for more than a few minutes, RabbitMQ queues rising, API error rates, Ceph warnings, compute hosts above a utilization threshold and failed...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Plan	Use growth trends and upcoming projects to forecast when capacity runs out, and account for procurement lead time.	Quotas that exceed physical capacity by a lot.

Flow: Control plane to Capacity / Capacity to Alerts / Alerts to Plan

Incident: Instances stuck in scheduling

Incident: *A swollen message queue slowed every operation.*

```
rabbitmq: conductor queue messages 412000, consumers 0
instances in BUILD for over 20 minutes: 311
nova-conductor service: down since 02:14
```

A conductor service had crashed, and requests piled up in its queue. Once restarted and the queue drained, builds resumed. The team added alerts on queue depth and consumers, and an automatic restart with a page for repeated crashes.

Common mistakes

- Watching only whether APIs respond
- No alert on queue depth
- Ignoring storage and network capacity
- Quotas that exceed physical capacity by a lot

Check yourself

What does a growing RabbitMQ queue suggest?

Answer: A stuck or slow consumer

What does overcommit ratio show?

Answer: Virtual resources compared to physical

Which command lists down compute services?

Answer: openstack compute service list

What else runs out besides CPU and memory?

Answer: Floating IPs, ports and storage pools

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: OpenStack and private cloud

The openstack client commands and checks for identity, compute, networking, volumes and troubleshooting a private cloud.

Sign in and check the basics

```
env | grep ^OS_
openstack token issue
openstack catalog list
openstack project list
openstack quota show
```

A working `token issue` proves Keystone and your credentials are good.

Compute

```
openstack server list
openstack server show app-9 -c status -c fault
openstack server create --flavor m1.small --image ubuntu --network private app-9
openstack server reboot app-9
openstack server delete app-9
openstack flavor list
openstack image list
openstack console log show app-9
```

Warning: *Servers in ERROR still count against your quota. Delete them.*

Scheduling and health

```
openstack compute service list
openstack hypervisor list
openstack hypervisor stats show
openstack resource provider usage show PROVIDER_ID
```

Capacity is per node. A big flavor can fail even when the total free memory is large.

Networking

```
openstack network list
openstack subnet list
openstack port list --server app-9
openstack security group list
openstack security group rule list web --long
openstack security group rule create --proto tcp --dst-port 22 --remote-ip 203.0.113.7/32 web
openstack floating ip create public
openstack server add floating ip app-9 203.0.113.45
```

Volumes

```
openstack volume list
openstack volume create --size 20 data
openstack server add volume app-9 data
openstack volume service list
```

Troubleshooting

Symptom	First look
HTTP 401	credentials, domain names, token expiry, clock
HTTP 403	role and policy, or quota
ERROR, no valid host	service state, per-node free resources, scheduler log
Cannot reach server	console log, security group, floating IP, router
Many services fail at once	RabbitMQ, database, disk space
Stuck state	compare hypervisor with database before reset

Follow one request through logs with its request id (`req-...`).

MARGA by EknathaLabs marga.eknathalabs.com

Quick health checks

```
openstack compute service list
openstack network agent list
openstack volume service list
openstack hypervisor stats show
openstack server show NAME -c status -c fault
```

Cautions

Danger: `openstack server delete` *removes the server and its ephemeral disk. Check for volumes and snapshots you need first.*

Warning: *A security group rule open to 0.0.0.0/0 on every port exposes every server that uses the group.*

Warning: *Evacuate a node only after it is fenced or powered off, or two copies of a server can run at once.*

Caution: *Back up the database before any upgrade. Many migrations cannot be reversed.*

MARGA by EknathaLabs
marga.eknathalabs.com